



Computer Science & Engineering, Khulna University

Data Mining

Text processing and Feature Extraction

Dr. Amit Kumar Mondal,
Associate Professor, CSE KU



References

1. Baeza-Yates, Ricardo, and Berthier Ribeiro-Neto. *Modern information retrieval*. Vol. 463. No. 1999. New York: ACM press, 1999.
2. **Natural Language Processing**, Jacob Eisenstein, 2013
3. Deerwester, Scott, et al. "Indexing by latent semantic analysis." *Journal of the American society for information science* 41.6 (1990): 391-407.
4. Jurafsky, Daniel. "**Speech and language processing**." (2024).
5. William M Campbell and Fred S Richardson. Discriminative keyword selection using support vector machines. In *NIPS*, pages 209–216. Citeseer, 2007.
6. Meng Yan, Ying Fu, Xiaohong Zhang, Dan Yang, Ling Xu, and Jeffrey D Kymer. Automatically classifying software changes via discriminative topic model: Supporting multi-category and cross-project. *Journal of Systems and Software*, pages 296–308, 2016 .
7. Sentiwordnet: <http://sentiwordnet.isti.cnr.it/>. June, 2019.
8. David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3:993–1022, 2003.



Information Retrieval Modeling

- Logical view of documents

Documents	
D1	I have implemented the Apriori algorithm to find frequent itemsets and association rules on my dataset and the <u>Apyori</u> library in Python gives me these results :
D2	this is the first time I am trying to code in python and I am implementing the Apriori algorithm. I have generated till 2-itemsets and below is the function I have to generate 2-Itemsets by combining the keys of the 1-itemset. How do I go about making this function generic? I mean, by passing the keys of a dictionary and the number of elements required in the tuple, the algorithm should generate all possible n-number(k+1) subsets using the keys. I know that Union on sets is a possibility, but is there a way to do union of tuples which is essentially the keys of a dictionary?
D3	I'm trying to implement the apriori algorithm . In one of the final steps I have two arrays of tuples generated from a list of products.



Latent Semantic Indexing

- "latent semantic indexing" (LSI) analysis ---- singular-value decomposition.
- A large matrix of term-document association data and **construct a "semantic" space** wherein **terms and documents that are closely associated** are placed near one another.
- Singular-value decomposition arranges of the space to **reflect the major associative patterns in the data**, and ignore the smaller, less important influences.
- Terms that did not actually appear in a document may still end up close to the document, if that is consistent with the major patterns of association in the data.
- Position in the space then serves as the new kind of semantic indexing
- retrieval proceeds by using the terms in a query to identify a point in the space, and documents in its neighborhood are returned to the user



Latent Semantic Indexing

Sample Term by Document matrix

	<i>access</i>	<i>document</i>	<i>retrieval</i>	<i>information</i>	<i>theory</i>	<i>database</i>	<i>indexing</i>	<i>computer</i>	REL	MATCH
Doc 1	x	x	x			x	x		R	
Doc 2				x*	x			x*		M
Doc 3			x	x*				x*	R	M

Query: "IDF in *computer-based information* look-up"



Latent Semantic Indexing

Such searching requires the following three criteria:

- 1. *Adjustable representational richness*. To represent the underlying semantic structure, we need a model with sufficient power.
 - We believe **hierarchical clusterings to be too restrictive**, since they allow no multiple or crossed classifications and have essentially only as many parameters as objects.
 - LSI can represent models whose power could be varied, as some compensation for choosing a perhaps inappropriate structure.
 - The most obvious class is dimensional models, like multidimensional scaling and factor analysis, where representational power can be controlled by choosing the number, k , of dimensions (i.e., k parameters per object).



Latent Semantic Indexing

- *2. Explicit representation of both terms and documents.*
 - The desire to represent both terms and documents simultaneously is more than esthetic.
 - In the proximity-based latent structure paradigm, retrieval proceeds by appropriately placing a new object corresponding to the query in the semantic structure and finding those documents that are close by.
 - One simple way to achieve appropriate placement is if terms, as well as documents, have positions in the structure.
 - Then a query can be placed at the centroid of its term points.
 - Thus for both elegance and retrieval mechanisms, we needed what are called two-mode proximity methods, that **start with a rectangular matrix** and construct **explicit representations of both row and column** objects.
 - One such method is **multidimensional unfolding**, in which both terms and documents would appear as points in a single space with similarity related monotonically to Euclidean distance.



Latent Semantic Indexing

- 3. *Computational tractability for large datasets.*
 - Many of the existing models require computation that goes up with N^4 or N^5 (where N is the number of terms plus documents).
 - with document sets that were at least in the thousands, models with efficient fitting techniques were needed.



Singular Value Decomposition

Singular Value Decomposition

- SVD represents both terms and documents as vectors in a space of choosable dimensionality, and the dot product or cosine between points in the space gives their similarity.
- In addition, a program was available that fit the model in time of order $N^2 \times k^3$

Latent Semantic Indexing



Technical Memo Example

Titles:

- c1: *Human machine interface for Lab ABC computer applications*
 - c2: *A survey of user opinion of computer system response time*
 - c3: *The EPS user interface management system*
 - c4: *System and human system engineering testing of EPS*
 - c5: *Relation of user-perceived response time to error measurement*
-
- m1: *The generation of random, binary, unordered trees*
 - m2: *The intersection graph of paths in trees*
 - m3: *Graph minors IV: Widths of trees and well-quasi-ordering*
 - m4: *Graph minors: A survey*

Latent Semantic Indexing



Terms	Documents								
	c1	c2	c3	c4	c5	m1	m2	m3	m4
<i>human</i>	1	0	0	1	0	0	0	0	0
<i>interface</i>	1	0	1	0	0	0	0	0	0
<i>computer</i>	1	1	0	0	0	0	0	0	0
<i>user</i>	0	1	1	0	1	0	0	0	0
<i>system</i>	0	1	1	2	0	0	0	0	0
<i>response</i>	0	1	0	0	1	0	0	0	0
<i>time</i>	0	1	0	0	1	0	0	0	0
<i>EPS</i>	0	0	1	1	0	0	0	0	0
<i>survey</i>	0	1	0	0	0	0	0	0	1
<i>trees</i>	0	0	0	0	0	1	1	1	0
<i>graph</i>	0	0	0	0	0	0	1	1	1
<i>minors</i>	0	0	0	0	0	0	0	1	1



Singular Value Decomposition

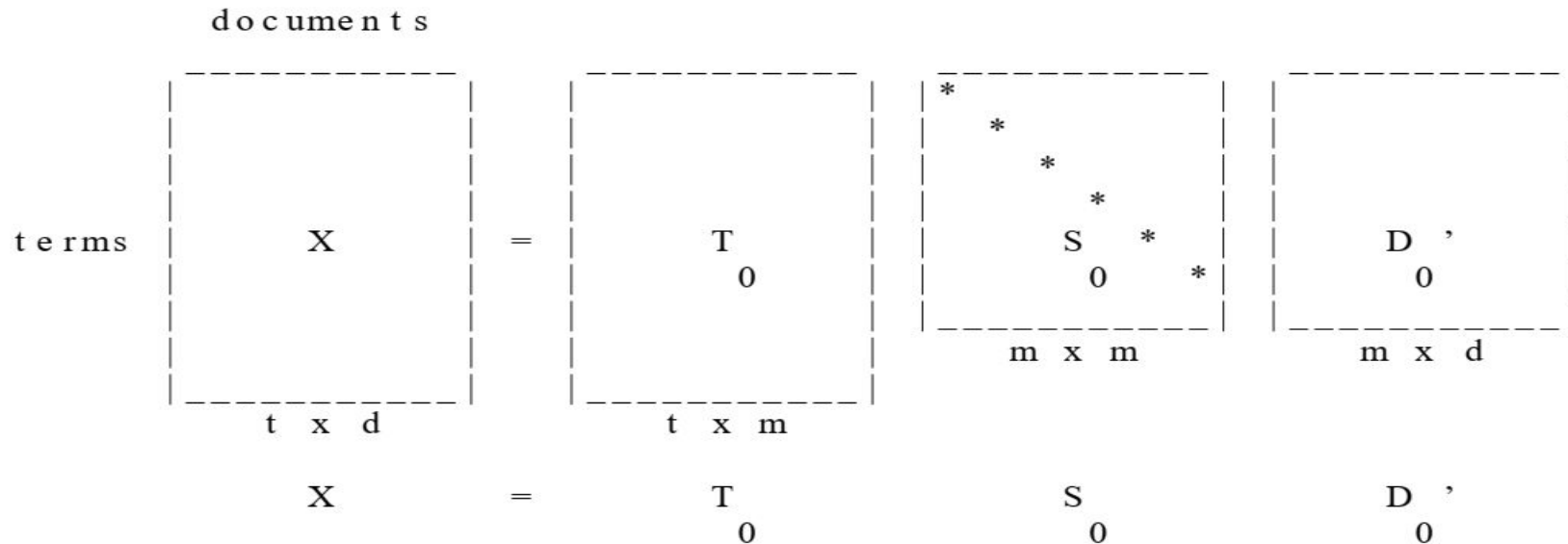
Any rectangular matrix, for example a $t \times d$ matrix of terms and documents, X , can be decomposed into the product of three other matrices:

$$X = T_0 S_0 D_0',$$

such that T_0 and D_0 have orthonormal columns and S_0 is diagonal. This is called the *singular value decomposition of X* . T_0 and D_0 are the matrices of *left* and *right singular vectors* and S_0 is the diagonal matrix of *singular values*.⁴ Singular value decomposition (SVD) is unique up to certain

row, column and sign permutations and by convention the diagonal elements of S_0 are constructed to be all positive and ordered in decreasing magnitude.

Singular Value Decomposition



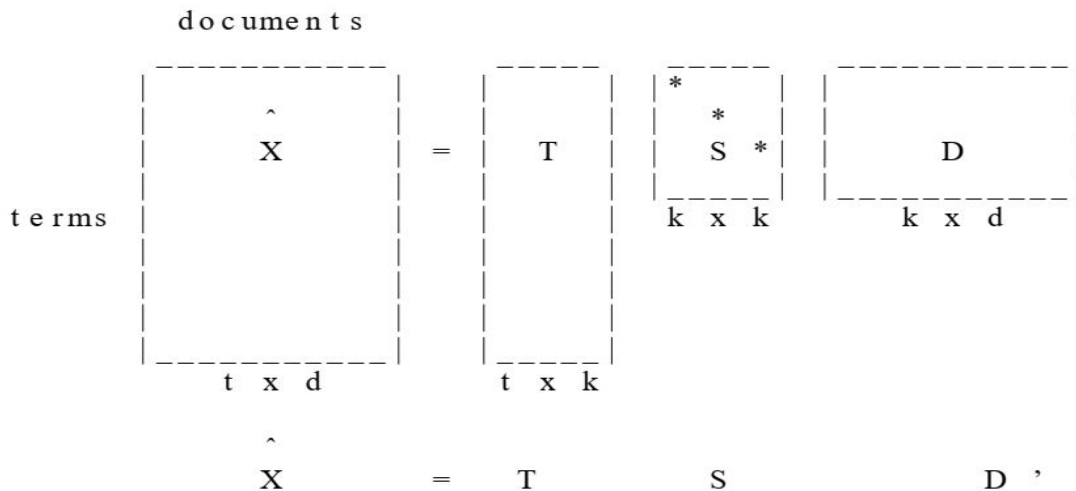


Singular Value Decomposition

S respectively. The result is a reduced model:

$$\hat{X} \approx TSD'$$

which is the rank- k model with the **best possible least-squares-fit to X** . It is presented in Figure 3, that we use to approximate our data.



Reduced singular value decomposition of the term x document matrix, $\hat{X}$.



Singular Value Decomposition

4.2.3.1 Comparing two terms. The dot product between two row vectors of $X\hat{h}at$ reflects the extent to which two terms have a similar pattern of occurrence across the set of documents. The matrix $X\hat{h}atX\hat{h}at'$ is the square symmetric matrix containing all these term-to-term dot products. Since S is diagonal and D is orthonormal It is easy to verify that:

$$X\hat{h}atX\hat{h}at'=TS^2T'$$

Note that this means that the i, j cell of $X\hat{h}atX\hat{h}at'$ can be obtained by taking the dot product **between the i and j rows of the matrix TS** . That is, if one considers the rows of TS as coordinates for terms, dot products between these points give the comparison between terms. Note that the relation between taking T as coordinates and taking TS as coordinates is simple since S is diagonal; the positions of the points are the same except that each of the axes has been stretched or shrunk in proportion to the corresponding diagonal element of S .



Singular Value Decomposition

4.2.3.2 *Comparing two documents.* The analysis for comparing two documents is similar, except that in this case it is the **dot product between two column vectors of the matrix $Xhihat$** which tells the extent to which two documents have a similar profile of terms. Thus the matrix $Xhihat'Xhihat$ contains the document-to-document dot products. The definitions of the matrices T , S and D again guarantee:

$$\cancel{Xhihat'Xhihat} = DS^2D'$$

Here the i, j cell of $Xhihat'Xhihat$ is obtained by taking the dot product between the i and j rows of the matrix DS . So one can consider rows of a DS matrix as coordinates for documents, and take dot products in this space. (Again note that the DS space is just a stretched version of the D space.)



Singular Value Decomposition

4.2.3.3 *Comparing a term and a document.* This comparison is different. Instead of trying to estimate the dot product between rows or between columns of X_{hihat} , the fundamental comparison between a term and a document is the value of an individual cell of X_{hihat} . X_{hihat} is defined in terms of matrices T , S and D . Repeating it here:

$$X_{hihat} = TSD'$$

The i, j cell of X_{hihat} is therefore obtained by taking the dot product between the i -th row of the matrix $TS^{1/2}$ and the j -th row of the matrix $DS^{1/2}$. Note that while the within comparisons (i.e., term-term or document-document) involve using rows of TS and DS for coordinates, the between comparison requires $TS^{1/2}$ and $DS^{1/2}$ for coordinates. That is, it is not possible to make a single configuration of points in a space that will allow both between and within comparisons. They will be similar however, differing only by a stretching or shrinking of the axes by a factor of $S^{1/2}$.



Singular Value Decomposition

4.2.4 Finding representations for pseudo-documents

appear in the original analysis. The new objects in both these examples are very much like the documents of the matrices, X and $X\hat{h}$, in that they present themselves as vectors of terms. It is for this reason that we call them *pseudo-documents*. In order to compare a query or pseudo-document, q , to other documents, we need to be able to start with its term vector X_q and derive a representation D_q that we can use just like a row of D in the comparison formulas of the preceding section. One criterion for such a derivation is that putting in a real document X_i should give D_i (at least when the model is perfect, i.e., $X=X\hat{h}$). With this constraint, a little algebra shows that:

$$D_q = X_q' T S^{-1}$$

Note that with appropriate rescaling of the axes, this amounts to placing the pseudo-document at the centroid of its corresponding term points. This D_q then is just like a row of D and, appropriately scaled by $S^{1/2}$ or S , can be used like a usual document's factor vector for making between or within comparisons, respectively.

Singular Value Decomposition



The example 12-term by 9-document matrix from Table 2 is presented below.

$$X = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \\ 11 \\ 12 \end{matrix} & \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \end{matrix}$$

Singular Value Decomposition



$$T_0 = \begin{bmatrix} 0.22 & -0.11 & 0.29 & -0.41 & -0.11 & -0.34 & 0.52 & -0.06 & -0.41 \\ 0.20 & -0.07 & 0.14 & -0.55 & 0.28 & 0.50 & -0.07 & -0.01 & -0.11 \\ 0.24 & 0.04 & -0.16 & -0.59 & -0.11 & -0.25 & -0.30 & 0.06 & 0.49 \\ 0.40 & 0.06 & -0.34 & 0.10 & 0.33 & 0.38 & 0.00 & 0.00 & 0.01 \\ 0.64 & -0.17 & 0.36 & 0.33 & -0.16 & -0.21 & -0.17 & 0.03 & 0.27 \\ 0.27 & 0.11 & -0.43 & 0.07 & 0.08 & -0.17 & 0.28 & -0.02 & -0.05 \\ 0.27 & 0.11 & -0.43 & 0.07 & 0.08 & -0.17 & 0.28 & -0.02 & -0.05 \\ 0.30 & -0.14 & 0.33 & 0.19 & 0.11 & 0.27 & 0.03 & -0.02 & -0.17 \\ 0.21 & 0.27 & -0.18 & -0.03 & -0.54 & 0.08 & -0.47 & -0.04 & -0.58 \\ 0.01 & 0.49 & 0.23 & 0.03 & 0.59 & -0.39 & -0.29 & 0.25 & -0.23 \\ 0.04 & 0.62 & 0.22 & 0.00 & -0.07 & 0.11 & 0.16 & -0.68 & 0.23 \\ 0.03 & 0.45 & 0.14 & -0.01 & -0.30 & 0.28 & 0.34 & 0.68 & 0.18 \end{bmatrix}$$



$$D_0 = \begin{bmatrix} 0.20 & -0.06 & 0.11 & -0.95 & 0.05 & -0.08 & 0.18 & -0.01 & -0.06 \\ 0.61 & 0.17 & -0.50 & -0.03 & -0.21 & -0.26 & -0.43 & 0.05 & 0.24 \\ 0.46 & -0.13 & 0.21 & 0.04 & 0.38 & 0.72 & -0.24 & 0.01 & 0.02 \\ 0.54 & -0.23 & 0.57 & 0.27 & -0.21 & -0.37 & 0.26 & -0.02 & -0.08 \\ 0.28 & 0.11 & -0.51 & 0.15 & 0.33 & 0.03 & 0.67 & -0.06 & -0.26 \\ 0.00 & 0.19 & 0.10 & 0.02 & 0.39 & -0.30 & -0.34 & 0.45 & -0.62 \\ 0.01 & 0.44 & 0.19 & 0.02 & 0.35 & -0.21 & -0.15 & -0.76 & 0.02 \\ 0.02 & 0.62 & 0.25 & 0.01 & 0.15 & 0.00 & 0.25 & 0.45 & 0.52 \\ 0.08 & 0.53 & 0.08 & -0.03 & -0.60 & 0.36 & 0.04 & -0.07 & -0.45 \end{bmatrix}$$



Singular Value Decomposition

$$X \approx X\hat{h}at = TSD'$$

$X \approx$	T	S	D'
0.22	-0.11	3.34	0.20 0.61 0.46 0.54 0.28 0.00 0.02 0.02 0.08
0.20	-0.07	2.54	-0.06 0.17 -0.13 -0.23 0.11 0.19 0.44 0.62 0.53
0.24	0.04		
0.40	0.06		
0.64	-0.17		
0.27	0.11		
0.27	0.11		
0.30	-0.14		
0.21	0.27		
0.01	0.49		
0.04	0.62		
0.03	0.45		

Multiplying out the matrices TSD' gives the following estimate of X , $X\hat{h}at$.



LSI Algorithm: SVD Composition

```
U, Sigma, VT = randomized_svd(self.train_data,  
                               n_components=self.f_number,  
                               n_iter=5,  
                               random_state=None)
```

```
# Get Singular values and Components
```

```
print("^ matrix")
```

```
print(Sigma)
```

```
self.S_matrix = np.diag(Sigma)
```

```
self.T_matrix = np.transpose(VT)
```

```
print("T matrix")
```

```
print(self.T_matrix.shape)
```

```
self.D_matrix = U
```

```
print("D matrix")
```

```
print(self.D_matrix.shape)
```



LSI Algorithm: Query Matrix D-q formation

```
xq = np.array(Xq) #matrix formed from query
print(xq.shape)
t_formed = np.transpose(xq)
comparison = np.transpose(t_formed@self.S_matrix)
concept_vectors = []
for vec in comparison:
    concept_vectors.append(sum(vec))
concept = concept_vectors.index(max(concept_vectors))
# print("Comparison")
# print(comparison)
xx = np.transpose(xq)@self.S_matrix@np.transpose(self.D_matrix)
self.Xihat = xx
s_inverse = np.linalg.inv(self.S_matrix)
print("s_inverse", s_inverse.shape)
# print(s_inverse)

self.D_q = np.transpose(np.transpose(xx)@self.T_matrix@s_inverse)
print("Centroid of Dq")
self.centroids = np.flip(self.D_q[concept])#self.D_q[concept]#
```



LSI Algorithm: Matching

```
vec = sum(self.centroids)
dd = np.transpose(self.Xihat)@self.Xihat
for i,d in enumerate(dd):
    dv = sum(d)
    result = abs(spatial.distance.cosine(dv, vec))
    if(result>0 and result<=threshold):
        matched.append(i)
```